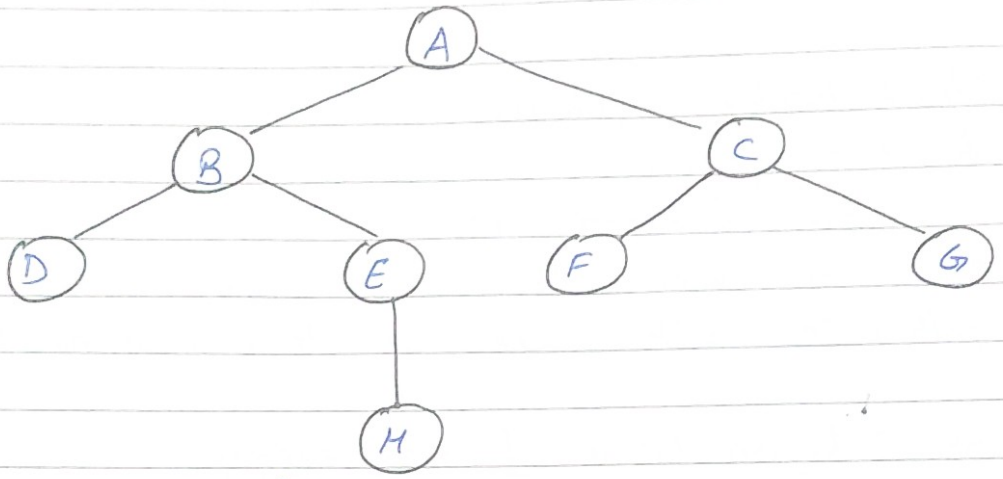
Lab-2b Depth First Search (DFS)

To implement the DFS algorithm to find a path from a start node to a goal node in a graph.



We need to find a path between A to H using the DFS algorithm.

Step 1: Create a graph as Dictionary

```
graph = {'A' : ['B', 'C'],
         'B' : ['D', 'E'],
         'C' : ['F', 'G'],
         'D' : [], 'E' : ['H'],
         'F' : [], 'G' : [], 'H' : []}
```

Same graph as BFS with the same nodes, children and leaves.
The data in both the traversals is the same, the change comes in the algorithm.

Step 2: Declare the function along with two variables

```
def dfs(start, goal):
    stack = [start]
    visited = []
```

the stack stores the path and not individual nodes just like BFS.

The variable queue and stack both are of same type i.e. list the only thing that changes is how we use them when we take queue we follow FIFO and when we take stack we follow LIFO.

Step 3 : Loop the stack and remove from end

```
while stack:
    path = stack.pop()
    node = path[-1]
```

stack.pop() takes from the end since LIFO
This is the only difference between the algorithms of DFS & BFS.

Step 4: Goal test

```
if node == goal:
    return path, visited
```

returns the path and visited nodes if the goal is reached

Step 5: Avoiding Revisiting

```
if node not in visited:
    visited.append(node)
```

In DFS this guard is protection against infinite descent. Without checking visited nodes, if there is any graph with cycle DFS will loop around forever.

Step 6: Expand in reverse

```
for n in reversed(graph[node]):
    stack.append(path + [n])
```

we use reversed function here for matching the paper trace where we generally go to the left child first then right.

Step 7: Return Failure

```
return None, visited
```

Failed when we don't find the node we are searching.

Step 8: Call and Print

```
p, v = dfs('A', 'H')
print("DFS path:", p)
print("DFS visited:", v)
```

For the problem graph the path from A to H using DFS is

DFS path: ['A', 'B', 'E', 'H']

DFS visited: ['A', 'B', 'D', 'E']